

05_PERFORMANCE_STANDARDS

- [SUOP ERP v1.0 — Enterprise Performance Standards](#)
 - [1. Purpose](#)
 - [2. Response Time Targets](#)
 - [3. Database Performance](#)
 - [4. API Performance](#)
 - [5. Frontend Performance](#)
 - [6. Caching Strategy](#)
 - [7. Redis Strategy](#)
 - [8. CDN Strategy](#)
 - [9. Load Testing](#)
 - [10. Stress Testing](#)
 - [11. Capacity Planning](#)
 - [12. Frontend Performance Monitoring](#)
 - [13. Database Performance Monitoring](#)
 - [14. Performance Regression Testing](#)
 - [15. Open Questions for Approval](#)
 - [Approval Block](#)

SUOP ERP v1.0 — Enterprise Performance Standards

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Approval
Date	2026-07-11
Approval Required	Project Owner

1. Purpose

This document defines performance standards for every layer of SUOP ERP v1.0. Every endpoint, every database query, every frontend page, and every background job must meet these standards. Performance is a feature — slow ERP = unusable ERP.

Hard rule: No endpoint ships without meeting its performance target. CI runs performance regression tests on every PR.

2. Response Time Targets

2.1 API Response Time Targets

Endpoint Type	p50	p95	p99	Hard Limit
Single record GET (/products/{id})	50ms	200ms	500ms	2s
List GET (/products? page=1&pageSize=25)	100ms	400ms	1s	3s
Filtered/aggregated list (dashboards)	200ms	800ms	2s	5s
Simple POST (/ products create)	100ms	400ms	1s	3s
Complex POST (PO create with 50 lines)	300ms	1s	2s	5s
Workflow transition (/pos/{id}/ approve)	200ms	800ms	2s	5s
	500ms	2s	5s	15s

Endpoint Type	p50	p95	p99	Hard Limit
Bulk operations (100 items)				
Long-running async (COA generation, recall)	200ms (return job ID)	500ms	1s	2s
Export to CSV (async)	200ms (return job ID)	500ms	1s	2s
Search (full-text)	200ms	800ms	2s	5s
Audit log query	200ms	800ms	2s	5s
Health check	5ms	20ms	50ms	100ms
Reference data GET	10ms	30ms	50ms	200ms (cached)

2.2 Frontend Performance Targets

Metric	Target	Measurement
Largest Contentful Paint (LCP)	<2.5s	Lighthouse on production
First Input Delay (FID)	<100ms	Lighthouse
Cumulative Layout Shift (CLS)	<0.1	Lighthouse
Time to First Byte (TTFB)	<600ms	Lighthouse
Total Blocking Time (TBT)	<300ms	Lighthouse

Metric	Target	Measurement
First Contentful Paint (FCP)	<1.8s	Lighthouse
Lighthouse Performance Score	≥90	Lighthouse
Page transition (SPA)	<300ms	Custom
API call to render	<500ms	Custom
Bundle size (initial)	<300KB gzipped	webpack-bundle-analyzer
Bundle size (per route)	<100KB gzipped	Per-route code splitting

2.3 Database Targets

Operation	Target	Hard Limit
Single row lookup by PK	<5ms	<50ms
Indexed range scan (1000 rows)	<20ms	<200ms
Filtered list (25 rows from 100k)	<50ms	<500ms
Filtered list (25 rows from 1M)	<100ms	<500ms
Join query (3 tables)	<50ms	<500ms
Aggregation (COUNT, SUM)	<100ms	<1s
	<500ms	<2s

Operation	Target	Hard Limit
Complex dashboard query		
Full-text search	<200ms	<1s
Bulk insert (500 rows)	<500ms	<2s
Bulk update (500 rows)	<1s	<5s
Migration (small)	<1s	<30s
Migration (large, with backfill)	<5 min	<30 min (zero-downtime required)

2.4 Background Job Targets

Job Type	Target Duration
Notification dispatch (single)	<2s
Audit log archival (10k records)	<5 min
Dashboard snapshot computation	<30s
Stock reorder check (full scan)	<2 min
Backup verification	<10 min
Mock recall drill	<60s (recall identification: <5s per acceptance criteria)
COA generation (single)	<3s per acceptance criteria
Compliance dashboard load	<2s per acceptance criteria

2.5 Real-Time Update Targets

Operation	Target
Notification delivery (in-app, polling)	<30s (poll interval)
Notification delivery (future: WebSocket)	<2s
Audit log visibility after action	<1s (same request)
Workflow state update propagation	<1s (same request)
Stock balance update after GRN post	<1s (same transaction)

3. Database Performance

3.1 Index Strategy (see Data Architecture §6)

- Every FK indexed
- Every query path indexed (verified via EXPLAIN ANALYZE in CI)
- Composite indexes for filter+sort patterns
- Partial indexes for soft-deleted tables
- Covering indexes (INCLUDE) for hot queries

3.2 Query Performance

- **N+1 prevention:** Prisma `include` for eager loading, DataLoader for batch loading
- **No `SELECT *` in production** (Prisma handles by default; raw queries must specify columns)
- **Cursor pagination for >100k row tables** (offset pagination slow at high offsets)
- **Pagination limit: max 200 rows per page** (prevents memory pressure)

3.3 Connection Pooling

- PgBouncer (transaction mode) between app and Postgres
- Pool size: 20-50 per app instance (tuned per workload)
- Connection lifecycle: idle timeout 30s, max lifetime 5 min
- Health check: pool utilization alarm at 80%

3.4 Slow Query Detection

- `log_min_duration_statement = 500ms` — logs slow queries
- Daily report of top 10 slowest queries
- Each query added to optimization backlog
- EXPLAIN ANALYZE auto-run on slow queries; plan stored

3.5 Query Plan Review

- Weekly job samples `pg_stat_statements` for top 20 expensive queries
- EXPLAIN ANALYZE plan stored
- Seq scans on tables >10k rows flagged
- Optimization PRs created from backlog

3.6 Table Maintenance

- **VACUUM:** Auto-vacuum tuned per table; manual `VACUUM ANALYZE` weekly on hot tables
- **ANALYZE:** After large data loads; ensures planner statistics current
- **REINDEX:** Monthly on high-write tables (using `REINDEX CONCURRENTLY` — no lock)
- **Bloat monitoring:** `pgstattuple` monthly; bloat >30% triggers REINDEX

3.7 Partitioning (see Data Architecture §7)

- Large tables partitioned by month
 - Old partitions dropped (instant) instead of DELETE
 - `pg_partman` for automatic partition management
-

4. API Performance

4.1 Response Caching

Cache layers: 1. **HTTP cache (CDN):** Static assets, reference data (1 hour TTL) 2. **Application cache (Redis):** Reference data, dashboard aggregations (5 min TTL) 3. **Database cache:** Postgres shared buffers (tuned by `shared_buffers`)

What to cache: - Reference data (countries, currencies, UOMs) — TTL 1 hour - Dashboard aggregations — TTL 5 min (stale-while-revalidate) - User permissions — TTL 60s - Product/supplier master data — TTL 5 min - Static assets — TTL 1 year (with hash-based filenames)

What NOT to cache: - Real-time data (stock balances, audit logs) - User-specific data (notifications, tasks) - Sensitive data (financial, PII)

4.2 ETag and Conditional Requests

- All GET responses include `ETag` (hash of response body)
- Clients send `If-None-Match: <etag>` on subsequent requests
- Server returns `304 Not Modified` if unchanged (no body — saves bandwidth)
- Reduces load for repeated polling (dashboards, notification checks)

4.3 Compression

- **gzip** for responses >1KB
- **Brotli** (preferred) for browsers that support it
- Disabled for already-compressed content (images, PDFs, videos)
- Content-Type whitelist: `application/json`, `text/html`, `text/css`, `application/javascript`

4.4 Request Batching

- Frontend batches independent API calls via `Promise.all`
- Backend does not support batch endpoints for unrelated resources (REST principles)

- Bulk endpoints exist for related bulk operations (bulk create, bulk update)

4.5 Idempotency Cache

- Idempotency keys stored in Redis (24h TTL) instead of DB
 - Repeat requests served from cache (<5ms response)
 - Reduces DB load for retry storms (client retrying on network error)
-

5. Frontend Performance

5.1 Bundle Size Budgets

```
// apps/web/budgets.config.js
module.exports = {
  bundle: {
    'initial': { max: 300 },      // KB gzipped
    'per-route': { max: 100 },   // KB gzipped
    'vendor': { max: 200 },      // KB gzipped
  },
}
```

- CI fails if bundle exceeds budget
- Bundle analyzed via `webpack-bundle-analyzer` in CI
- Tree-shaking verified for unused exports

5.2 Code Splitting

- **Route-level:** Each route lazy-loaded (`dynamic(() => import(...))`)
- **Component-level:** Heavy components (data tables, charts) lazy-loaded
- **Library-level:** Large libs (`chart.js` , `moment` replaced with `date-fns`) code-split
- **Vendor-level:** React, React-DOM in vendor chunk (long-term cache)

5.3 Image Optimization

- **Next.js Image component** for all images
- **WebP/AVIF** format (smaller than JPEG/PNG)
- **Responsive srcset** for different screen sizes
- **Lazy loading** for below-the-fold images
- **Blur placeholder** for above-the-fold images

5.4 Font Loading

- **Subset fonts** (Latin + Devanagari only)
- **Preload critical fonts** (`<link rel="preload">`)
- **Font-display: swap** (text visible during font load)
- **Self-host fonts** (no Google Fonts request — privacy + perf)

5.5 React Performance

- `useMemo` / `useCallback` for expensive computations and stable references
- `React.memo` for pure components that re-render unnecessarily
- **Virtual scrolling** for long lists (>100 items) via `@tanstack/react-virtual`
- **Debounced inputs** for search boxes (300ms debounce)
- **Optimistic updates** for mutations (UI updates before API response)

5.6 React Query Performance

- **Stale time:** 30s for transactional data, 5 min for master data, 1 hour for reference data
- **Cache time:** 5 min (garbage collected if not used)
- **Refetch on focus:** Yes (for transactional data), No (for reference data)
- **Refetch interval:** 30s for notifications, 5 min for dashboards
- **Pagination:** `useInfiniteQuery` for infinite scroll; prefetch next page
- **Optimistic updates:** `onMutate` rolls back on error

5.7 SSR vs CSR

- **SSR (Server-Side Rendering):** Public pages (login, marketing), SEO-required pages
 - **SSG (Static Site Generation):** Reference data pages, documentation
 - **CSR (Client-Side Rendering):** Authenticated dashboard pages (data is user-specific)
 - **ISR (Incremental Static Regeneration):** Help articles, changelog (revalidate every hour)
-

6. Caching Strategy

6.1 Cache Layers

Browser Cache (HTTP) → CDN Cache → App Cache (Redis) → DB Cache (Postgres shared buffers) → DB

6.2 Cache Invalidation Patterns

TTL-based (time): - Reference data: 1 hour - Master data: 5 min - Dashboard aggregations: 5 min (stale-while-revalidate) - User permissions: 60s

Event-based (write invalidates cache): - Product updated → invalidate `product:{id}`, `products:list` - Supplier updated → invalidate `supplier:{id}`, `suppliers:list` - Permission changed → invalidate `user:{id}:permissions`

Version-based: - Reference data updates bump version; clients refetch

6.3 Cache-Aside Pattern

```
async function getProduct(id: string): Promise<Product> {  
  const cacheKey = `product:${id}`
```

```
// Try cache
const cached = await redis.get(cacheKey)
if (cached) return JSON.parse(cached)
// Cache miss → DB
const product = await prisma.product.findUnique({ where:
  { id } })
if (!product) throw new NotFoundError()
// Cache for 5 min
await redis.setex(cacheKey, 300, JSON.stringify(product))
return product
}
```

6.4 Cache Stampede Prevention

- **Probabilistic early expiration:** Refresh cache before TTL expires (5% chance per request in last 10% of TTL)
- **Lock-based:** Only one request refreshes cache; others wait
- **Single-flight:** `Promise` shared across concurrent requests for same key

6.5 Cache Warming

- On deploy, warm cache for hot endpoints (reference data, top 100 products, top 100 suppliers)
- Background job every 5 min refreshes dashboard aggregations
- Reduces cold-start latency

7. Redis Strategy

7.1 Redis Use Cases

Use Case	Data Type	TTL	Memory Impact
Application cache (objects)	String (JSON)	5 min - 1 hour	High

Use Case	Data Type	TTL	Memory Impact
Session cache (user permissions)	String (JSON)	60s	Medium
Rate limiting counters	String (int)	60s	Low
Idempotency keys	String (JSON)	24 hour	Medium
JWT blocklist	String	Token TTL	Low
Distributed locks	String + NX	30s	Low
Job queue (BullMQ)	List + Hash	Persistent	Medium
Pub/Sub (future: WebSocket)	Pub/Sub	—	Low
Leaderboards (future: gamification)	Sorted Set	Persistent	Low

7.2 Redis Configuration

- **Maxmemory policy:** `allkeys-lru` (cache mode — old keys evicted)
- **Persistence:** RDB snapshot every 5 min + AOF (append-only file) for durability
- **Replication:** Single replica for read scaling (future)
- **TLS:** Required in production
- **Auth:** ACL-based (separate users for cache vs job queue)

7.3 Memory Management

- **Max memory:** 2GB per instance (sufficient for cache + queues)
- **Eviction policy:** LRU (least recently used)
- **Monitoring:** Memory usage, eviction rate, hit rate
- **Alert:** Memory >80% → scale up; >95% → critical

7.4 Cache Hit Rate Target

- **Reference data:** >99% (very stable, long TTL)
- **Master data:** >95% (5 min TTL, frequent reads)
- **Dashboard aggregations:** >90% (5 min TTL, periodic refresh)
- **User permissions:** >85% (60s TTL, periodic refresh)

Hit rate <80% triggers review (cache key strategy, TTL tuning).

8. CDN Strategy

8.1 CDN Use

- **Static assets:** JS, CSS, images, fonts (1 year cache, hash-based filenames)
- **Reference data API:** Cached at edge (1 hour TTL)
- **Public pages:** Login, marketing, documentation
- **Not cached:** Authenticated API responses, user-specific data

8.2 CDN Provider

- **Cloudflare** (default) — generous free tier, global PoPs, includes DDoS protection
- **Alternative:** AWS CloudFront (deeper AWS integration)

8.3 CDN Configuration

- **Cache key:** URL + Accept-Encoding (ignore query params for static assets)

- **Cache behavior:** Cache on GET/HEAD only; bypass for POST/PUT/DELETE
- **TTL:** 1 year for static assets with hash filenames; 1 hour for reference API
- **Purge:** API-driven purge on reference data updates
- **Compression:** Brotli (preferred) or gzip
- **Image optimization:** Cloudflare Images or Polish (resize, convert to WebP)

8.4 Cache Invalidation

- Hash-based filenames auto-invalidate on content change
 - Reference data updates trigger programmatic purge via CDN API
 - Full purge only for emergency (rare; affects all users)
-

9. Load Testing

9.1 Load Testing Tool

- **k6** (Grafana Labs) — scriptable in JavaScript, integrates with Grafana
- **Alternative:** Locust (Python), JMeter (heavy)

9.2 Load Test Scenarios

Scenario 1 — Normal Load: - 100 concurrent users - Read-heavy (90% GET, 10% POST) - Run for 10 min - Pass criteria: p95 <400ms, error rate <0.1%

Scenario 2 — Peak Load: - 500 concurrent users - Mix: 80% GET, 15% POST, 5% PATCH - Run for 30 min - Pass criteria: p95 <1s, error rate <1%

Scenario 3 — Stress Test: - Ramp from 100 → 2000 concurrent users over 30 min - Find breaking point - Document max throughput + latency at saturation

Scenario 4 — Spike Test: - 100 users baseline → 1000 users spike for 1 min → back to 100 - Verify auto-scaling responds - Verify no error storm

Scenario 5 — Soak Test: - 200 concurrent users for 8 hours - Detect memory leaks, connection leaks - Verify stable performance over time

9.3 Load Test Targets

Endpoint	Target Throughput	Target p95
GET /products (list)	500 req/s	<400ms
GET /products/{id}	1000 req/s	<200ms
POST /products	50 req/s	<400ms
GET /audit-logs (cursor)	200 req/s	<800ms
POST /purchase-orders (50 lines)	10 req/s	<1s
POST /auth/login	50 req/s	<500ms

9.4 Load Test Schedule

- **Every PR:** Smoke load test (50 users, 1 min)
- **Nightly on staging:** Peak load test (500 users, 30 min)
- **Weekly:** Stress test to find breaking point
- **Monthly:** Soak test (8 hours)
- **Quarterly:** Full load test suite + capacity review

10. Stress Testing

10.1 Stress Test Goals

- Find system breaking point
- Identify bottlenecks (CPU, memory, DB, Redis, network)
- Verify graceful degradation (not crash)
- Verify auto-scaling responds
- Document max capacity for capacity planning

10.2 Stress Test Procedure

1. Start with normal load (100 users)
2. Ramp +100 users every 5 min
3. Monitor: latency, error rate, CPU, memory, DB connections, Redis memory
4. Continue until error rate >5% or latency p95 >5s
5. Document breaking point + bottleneck
6. Scale bottleneck + retest

10.3 Graceful Degradation

When overloaded, system should: - Return 503 Service Unavailable (not crash) - Maintain functionality for authenticated users (rate limit unauthenticated) - Shed load via rate limiting (429) before running out of resources - Not corrupt data (transactions either complete or roll back)

11. Capacity Planning

11.1 Capacity Metrics

Resource	Current Utilization	Headroom	Scale Trigger
CPU (backend)	<50%	2x	>70% sustained 5 min → scale up
Memory (backend)	<60%	1.5x	>80% sustained 5 min → scale up
DB connections	<50%	2x	>70% → add app instances or increase pool
DB CPU	<50%	2x	>70% → scale up DB instance

Resource	Current Utilization	Headroom	Scale Trigger
DB storage	<70%	1.5x	>80% → provision more storage
Redis memory	<50%	2x	>70% → scale up or evict more
Redis CPU	<50%	2x	>70% → scale up or read replica

11.2 Growth Assumptions

Metric	Current (Year 1)	Year 2	Year 3
Tenants	5	20	50
Users per tenant	50	100	200
Total users	250	2,000	10,000
Transactions/day	10k	50k	200k
Audit logs/month	100k	500k	2M
DB size	5 GB	25 GB	100 GB
File storage	10 GB	50 GB	200 GB

11.3 Scaling Strategy

Vertical scaling (bigger instance): - DB: Scale up to larger RDS instance (CPU + memory) - Redis: Scale up to larger ElastiCache instance - App: Scale up to larger ECS task (more CPU + memory)

Horizontal scaling (more instances): - App: Add ECS tasks behind ALB (stateless — easy) - DB: Read replica for read-heavy workloads (reports, dashboards) - Redis: Read replica for cache (future)

When to scale: - CPU >70% sustained → scale horizontally (app) or vertically (DB) - Memory >80% sustained → scale vertically - DB connections >70% → scale horizontally (app) with PgBouncer - Storage >80% → provision more (automatic with managed services)

11.4 Auto-Scaling Rules

Backend (ECS): - Scale out: CPU >70% for 5 min (add 1 task) - Scale in: CPU <30% for 15 min (remove 1 task) - Min: 2 tasks (HA) - Max: 10 tasks (cost control)

Database (RDS): - Manual scaling only (vertical) — auto-scaling not recommended for DBs - Alert at 70% CPU; ops reviews weekly

Redis (ElastiCache): - Manual scaling only - Alert at 70% memory

12. Frontend Performance Monitoring

12.1 Real User Monitoring (RUM)

- **Sentry Performance** (or Datadog RUM) on production frontend
- Captures: page load time, route transitions, API call duration, errors
- Aggregated by: route, browser, country, device
- Alerts on regression: LCP >4s for 5% of users

12.2 Synthetic Monitoring

- **Checkly** runs synthetic transactions every 5 min:
 - Login → dashboard load → create PO → logout
 - Login → NCR dashboard → create NCR → logout
 - 10 critical paths monitored
- Alerts on failure or slowdown

12.3 Lighthouse CI

- Lighthouse runs on every PR (against preview deployment)
- Performance score <80 blocks merge
- Lighthouse runs weekly on production
- Trends tracked over time

13. Database Performance Monitoring

13.1 PostgreSQL Metrics

- **Connections:** active, idle, waiting
- **Queries per second** by type
- **Query latency** p50, p95, p99
- **Transaction duration** p50, p95, p99
- **Lock waits** (count + duration)
- **Cache hit ratio** (`blks_hit / (blks_hit + blks_read)`)
- **Index usage** (`pg_stat_user_indexes`)
- **Table bloat** (`pgstattuple`)
- **Slow queries** (>500ms logged)
- **Deadlocks**
- **Replication lag** (if replica exists)

13.2 Query Performance

- `pg_stat_statements` extension enabled
- Weekly report: top 20 queries by total time, by mean time, by calls
- Slow queries added to optimization backlog
- EXPLAIN ANALYZE plans stored for review

13.3 Index Maintenance

- Monthly: `pg_stat_user_indexes` review
 - Unused indexes (`idx_scan = 0` over 30 days) → candidates for removal
 - Bloat >30% → `REINDEX CONCURRENTLY`
 - New indexes verified via EXPLAIN ANALYZE in CI
-

14. Performance Regression Testing

14.1 Regression Test Suite

- 50 critical endpoints with target latency
- Run on every PR (smoke — 5 endpoints)
- Run nightly on staging (full — 50 endpoints)
- Run weekly on production-equivalent environment (full)

14.2 Regression Detection

- Latency >20% slower than baseline → fail
- Throughput >20% lower than baseline → fail
- Baseline updated monthly (intentional updates documented)

14.3 Performance Budget Enforcement

- Bundle size budget per route (CI fails if exceeded)
- Database query count budget per endpoint (CI fails if exceeded)
- Memory usage budget per endpoint (CI fails if exceeded)

15. Open Questions for Approval

#	Decision	Recommendation	Alternatives
Q-P1	API p95 target for list endpoints	400ms	200ms (more strict), 800ms (lenient)
Q-P2	Lighthouse performance score	≥90	≥80, ≥95
Q-P3	Bundle size budget (initial)	300KB gzipped	200KB, 500KB
Q-P4		5 min	1 min, 15 min

#	Decision	Recommendation	Alternatives
	Cache TTL for master data		
Q-P5	Load test frequency	Nightly on staging	Weekly
Q-P6	Stress test frequency	Weekly	Monthly
Q-P7	Auto-scaling for backend	Yes (ECS)	Manual scaling
Q-P8	RUM tool	Sentry Performance	Datadog RUM
Q-P9	Synthetic monitoring	Checkly	Pingdom
Q-P10	Performance regression threshold	20% slower	10%, 30%

Approval Block

Approved by: _____ **Date:** _____

End of Enterprise Performance Standards Document